SY B.Tech. - ARTIFICIAL INTELLIGENCE AND MACHINE LEARNING

PRACTICAL ASSIGNMENT-8

1. Student Details

StudentName	Homesh Sawarkar
PRN	202501110069
Division	A
Batch	A-2
Roll No.	202501110069
Date ofSubmission	24-10-2026

2. Practical Details

Practical No.	8
Practical Title	Implementation of Exception Handling in C++ (try, throw and catch)
Date Performed	
Faculty Name	Dr. Khushal Khairnar

3. Problem Statement

Writea C++ programto implement **exception handling** using the keywords **try**, **throw** and **catch**. The program should handle division by zero, use **multiple catch blocks** to handle different types of exceptions (for example an invalid age) and use a **catch-all handler** catch(...) for any other exception.

4. Objectives

1. To understand what an exception is and why exception handling is needed.
2. To use the try, throw and catch keywords.
3. To use multiple catch blocks for different data types.
4. To use catch(...) to catch any type of exception.
5. To make a program continue normally instead of crashing when an error occurs.

5. Expected Learning Outcomes

- Write atryblockandthrowan exceptionwhen an error condition occurs.
- Write catch blocks that catch the exception and show a proper error message.
- Handle different exception types with multiple catch blocks.
- Understand the flow of the program when an exception is thrown or not thrown.

6. Theory / Concept

Exception: An exception is an error that happens while the program is running (run time error), for example dividing a number by zero, entering a wrong value, or opening a file that does not exist. If we do not handle it, the program may stop suddenly (crash).

Exception handling: It is a way of catching run time errors and dealing with them properly, so that the program does not crash and can show a useful message or continue with the remaining work. It separates the normal code from the error-handling code, which makes the program clean and easy to understand. C++ uses three keywords for exception handling:

Keyword	Meaning	Example
try	The block of code in which an error may occur is written here.	try { ... }
throw	Sends (raises) the exception when an error is found.	throw "Error";
catch	Receives the exception and handles it.	catch (const char *msg) { ... }

How it works: The statements inside the try block run normally. When a throw statement runs, the control immediately leaves the try block (the remaining statements of the try block are skipped) and goes to the catch block whose type matches the thrown value. After the catch block finishes, the program continues with the statements after the last catch block. If no error occurs, the catch blocks are skipped.

Multiple catch blocks: One try block can have many catch blocks, each for a different data type (int, const char*, double, etc.). The catch block that matches the type of the thrown value is executed. Example: throw 15; is caught by catch (int x), and throw "text"; is caught by catch (const char *msg). **Catch-all handler:** catch(...) catches an exception of any data type. It is written after all other catch blocks and is used when we do not know the type of exception.

Uncaught exception: If an exception is thrown and no matching catch block is found, the program is terminated by the function terminate() (the program stops).

Standard exceptions: C++ also provides the standard class exception (in the header <exception>) and classes such as out_of_range and bad_alloc, which are used with catch (exception &e) and e.what() to get the error message.

7. Algorithm / Pseudocode

1. START
2. Read two numbers a and b and the age of the user.
3. Division: inside a try block, if b is 0 then throw the message "Division by zero is not allowed"; otherwise display a / b.
4. Write a catch block for const char* that displays the error message.
5. Age check: inside a try block, if age is greater than 100 then throw the message "Age is not valid".
6. If age is less than 18 then throw the age (int); otherwise display that the age is eligible.
7. Write two catch blocks: catch(int x) for the age less than 18 and catch(const char *msg) for the invalid age.
8. Catch all: inside a try block throw the value 1.5 (double) and catch it with catch(...).
9. Display "Program ended normally".
10. STOP

08. Technology / Tools / Libraries Used

Technology / Tool	Purpose
Programming Language	C++
IDE / Platform	Visual Studio Code (VS Code)
Libraries / Packages	iostream

09. Implementation / GitHub Repository

Particular	Details
GitHub Repository URL	
Repository Name	Assignment8
Branch Used	main
Commit ID / Latest Commit	Add file via upload
Dateof Last Commit	24-10-2026

10. Test Cases / Input and Output

Test Case	Input	ExpectedOutput	Actual Output	Status
1	10 2 25	Result = 5; Age 25 is eligible; Some unknown exception caught; Program ended normally	same as expected	pass
2	10 0 15	Error: Division by zero is not allowed; Error: age 15 is less than 18, not eligible; Some unknown exception caught; Program ended normally Result = 2;	same as expected	pass
3	8 4 150	Error: Age is not valid; Some unknown exception caught; Program ended normally	same as expected	pass



11. Results / Observations

TheC++ program wasimplemented in Visual Studio Code. When the second number was 0, the throw statement sent a message and the catch block displayed it, so the program did not crash. In the age check, two different catch blocks were used: the integer exception was caught by catch(int x) and the text exception by catch(const char *msg). The catch(...) block caught the double value, which has no separate catch block. In every test case the program continued after handling the exception and printed 'Program ended normally'. All test cases gave the expected output.

12. Conclusion

The practical successfully demonstrated exception handling in C++ using try, throw and catch. It showed that run time errors like division by zero can be handled safely, that multiple catch blocks can handle different exception types, and that catch(...) can catch any exception, so the program keeps running without crashing.

13. Assignment Questions

Q1.What is an exception? Explain try, throw and catch.

Answer: An exception is a run time error, such as division by zero, that can stop the program. In C++ it is handled with three keywords. The try block contains the code where an error may occur. The throw statement raises (sends) the exception when an error is found. The catch block receives the thrown exception and handles it, for example by showing an error message. After handling, the program continues normally.

Q2. What are multiple catch blocks? What is catch(...)?

Answer: A single try block can be followed by many catch blocks, each written for a different data type, and the one whose type matches the thrown value runs. For example, catch(int x) handles an int and catch(const char *msg) handles a text message. The catch(...) block is a catch-all handler that can catch an exception of any type; it is written at the end, after all the other catch blocks.

Q3. What happens if an exception is thrown but not caught?

Answer: If no matching catch block is found, the exception is not handled and the program calls the function terminate(), which stops the program abnormally. To avoid this, a catch(...) block can be added to catch all types of exceptions.

Q4. What is the benefit of exception handling compared to normal if-else error checking?

Answer: With exception handling, the normal code and the error-handling code are kept separate, so the program is cleaner and easier to read. An exception can be thrown from deep inside a function and caught at a suitable place, instead of checking and returning an error code after every call. It also prevents the program from crashing suddenly, because every error can be handled in a proper way.

14. Student Declaration

Ihereby declarethatthe abovepractical work has been completed by me and that the implementation, analysis, and answers are my own work

Student Name	PRN	Signature
Homesh Sawarkar	202501110069	Homesh A.S